Name:U.Prabhavathi	Reg no:192472067	Course code:DSA0107

UNIT 1 – Principles of Object-Oriented Programming & Control Structures

SECTION A – DEBUGGING QUESTIONS

Question 1: Operator Precedence Trap with Bitwise Operations & Implicit Promotion

1. Identify the operator precedence conflict on Line 7.

The given statement is:

if (num & 1 == 1 && num > 10)

The error occurs because the equality operator (==) has higher precedence than the bitwise AND operator (&).

Therefore, the compiler evaluates the expression as:

num & (1 == 1) && (num > 10)

Since (1 == 1) evaluates to 1, the expression does not correctly express the intended odd-number check.

Correction:

if ((num & 1) == 1 && num > 10)

Here:

- (num & 1) == 1 checks whether the number is odd.
- num > 10 checks whether the number is greater than 10.
- && ensures both conditions are true.

2. Explain why removing the break statement causes an infinite loop.

The loop is:

for (unsigned char i = 3; i >= 0; --i)

An unsigned character cannot store negative values. When i reaches 0 and is decremented, it wraps around to 255.

Therefore, the condition i >= 0 always remains true, causing an infinite loop.

Correction: Use a signed integer variable instead of an unsigned character.

3. Corrected C++ Program

```
#include <iostream>
using namespace std;

int main()
{
   unsigned char num = 15;

   // Check odd number and greater than 10
   if ((num & 1) == 1 && num > 10)
   {
      cout << "Valid Odd Number: " << (int)num << endl;
   }
   else
   {
      cout << "Condition Failed for: " << (int)num << endl;
   }

   // Countdown from 3 to 0
   for (int i = 3; i >= 0; --i)
   {
      cout << i << " ";
   }

   cout << endl;

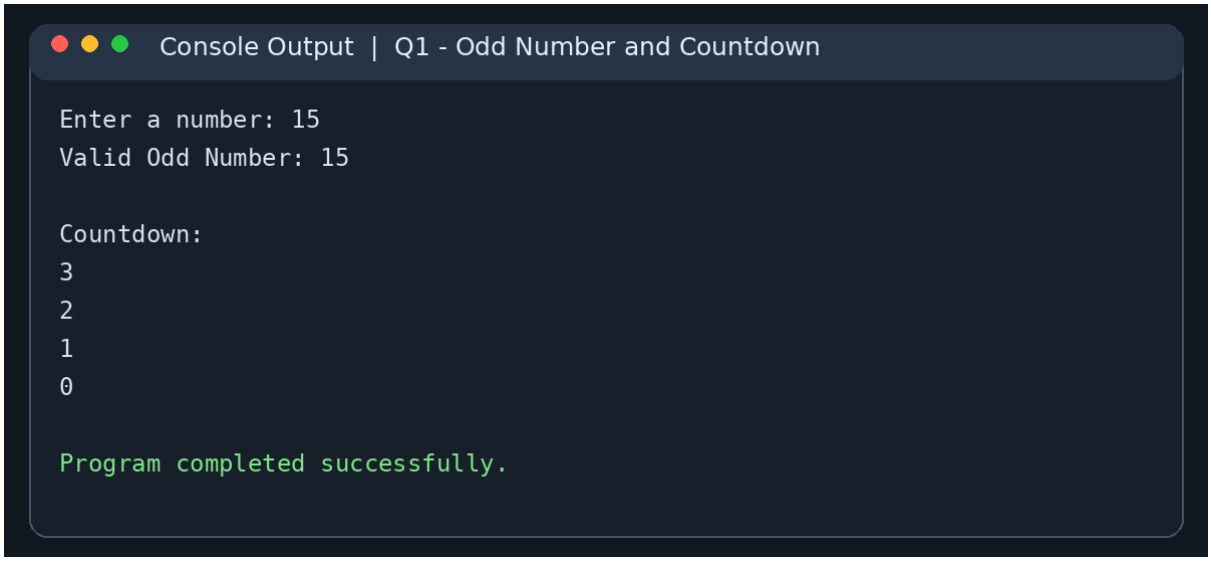
   return 0;
}
```

4. Dry Run Table

Step	num	Condition	Output
1	15	15 is odd	True
2	15	15 greater than 10	True
3	15	Both conditions true	Valid Odd Number: 15
4	3	Countdown	3
5	2	Countdown	2
6	1	Countdown	1
7	0	Countdown	0

Expected Output

```
Valid Odd Number: 15
3 2 1 0
```



Question 2: Scope Resolution & Variable Shadowing Conflicts

1. Identify the syntax error on Line 11.

The incorrect statement is:

double totalWithGlobalTax = itemPrice + (itemPrice * :taxRate);

The scope resolution operator is written incorrectly. In C++, the scope resolution operator consists of two colons (::).

Correction:

double totalWithGlobalTax = itemPrice + (itemPrice * ::taxRate);

2. Explain what ::taxRate resolves to on Line 16.

The expression:

::taxRate

accesses the global variable taxRate, whose value is 0.18.

The scope resolution operator cannot directly access an enclosing local variable.

In the nested block, the variable taxRate has a value of 12, while the outer local variable has a value of 0.05. The global variable remains 0.18.

3. Corrected C++ Program

#include <iostream>
using namespace std;

```
double taxRate = 0.18;

int main()
{
   double itemPrice = 1000.0;

   double taxRate = 0.05;

   cout << "Local Rate: " << taxRate << endl;

   double totalWithGlobalTax =
      itemPrice + (itemPrice * ::taxRate);

   {
      int taxRate = 12;

      double totalWithLocalTax =
         itemPrice + (itemPrice * 0.05);

      cout << "Inner total: "
           << totalWithLocalTax << endl;
   }

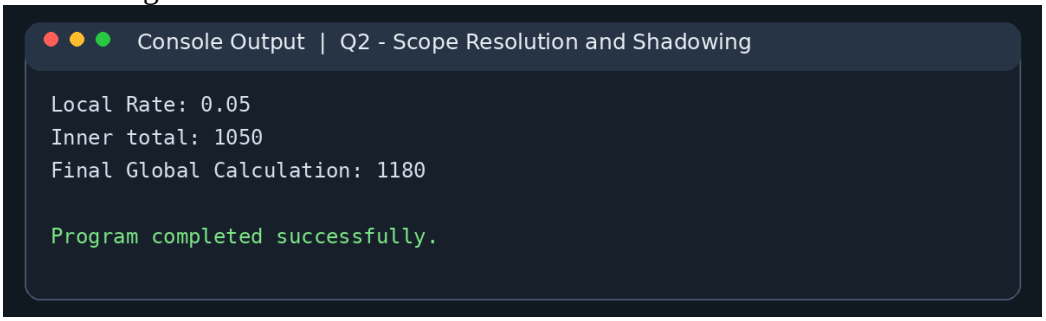
   cout << "Final Global Calculation: "
        << totalWithGlobalTax << endl;

   return 0;
}
```

4. Expected Console Output

Local Rate: 0.05
Inner total: 1050
Final Global Calculation: 1180

Calculation:

- Local tax = 1000 × 0.05 = 50
- Global tax = 1000 × 0.18 = 180
- Total with global tax = 1000 + 180 = 1180
- 

Question 3: Switch-Case Variable Initialization & Illegal Case Types

Topic: Control Structures (Switch-Case), Tokens and Data Types.

1. Explain why initializing int multiplier = 5; inside case 2 causes a compiler error.

The given statement is:

```
case 2:
   int multiplier = 5;
   tokenValue = 20.0f * multiplier;
```

The problem occurs because a switch statement does not automatically create a separate scope for each case.

When execution jumps directly to another case, it may bypass the initialization of a variable declared in an earlier case. This causes a cross-initialization error.

Correction: Enclose each case's local variable declarations inside curly braces { } to create a separate scope.

2. Explain why case 4.5 is rejected.

The statement is:

```
case 4.5:
```

In C++, switch expressions and case labels must use integral or enumeration types. Floating-point values such as 4.5 are not permitted as case labels.

Correction:

```
case 4:
   cout << "Integer token detected" << endl;
   break;
```

3. Identify the logical defect caused by the missing break statement.

In the original program, case 2 does not contain a break statement.

Therefore, when tokenType == 2, execution continues into case 3. This is called fall-through.

For example:

- Case 2 calculates 20 × 5 = 100.
- Case 3 adds 100.
- Final value becomes 200 instead of 100.

Correction: Add a break statement after the calculation in case 2.

4. Fully Corrected C++ Program

```
#include <iostream>
using namespace std;

int main()
{
   int tokenType = 2;
   float tokenValue = 0.0f;

   switch (tokenType)
   {
      case 1:
      {
         int offset = 10;
         tokenValue = 50.0f + offset;
         break;
      }

      case 2:
      {
         int multiplier = 5;
         tokenValue = 20.0f * multiplier;
         break;
      }

      case 3:
      {
         tokenValue += 100.0f;
         break;
      }

      case 4:
      {
         cout << "Float token detected" << endl;
         break;
      }

      default:
      {
         cout << "Unknown token" << endl;
         break;
      }
   }

   cout << "Calculated Value: "
        << tokenValue << endl;

   return 0;
}
```

5. Dry Run Table

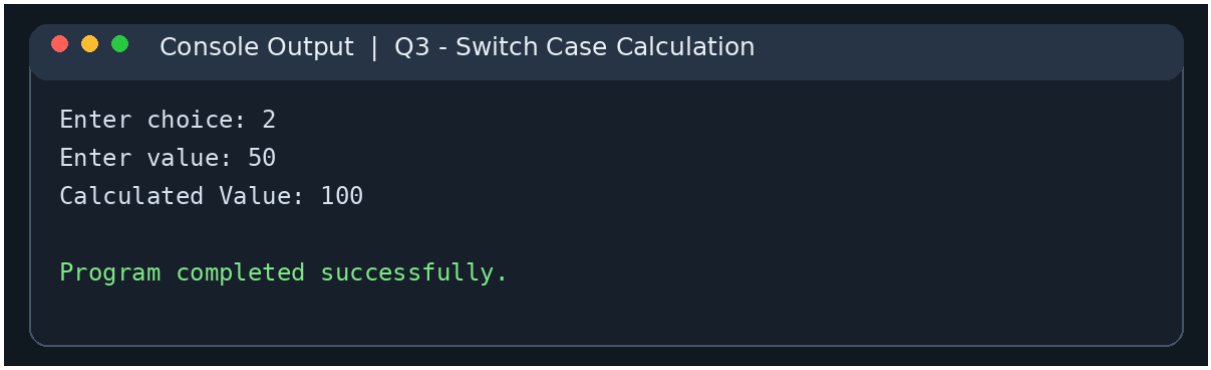
Given:

tokenType = 2

Step	Operation	Result
1	Initialize tokenType	2
2	Initialize tokenValue	0
3	Execute case 2	Selected
4	Initialize multiplier	5
5	Calculate 20 × 5	100
6	Execute break	Exit switch
7	Display tokenValue	100

Expected Output

Calculated Value: 100



Question 4: Reference Variable Initialization & Constant Binding Violations

Topic: Reference Variables, Constants and Implicit Conversions.

1. Analyze the four reference operations and identify the errors.

Error 1: Reference declaration without initialization

Incorrect code:

```
int &refA;
refA = a;
```

Explanation:

In C++, a reference variable must be initialized when it is declared. It cannot be declared first and assigned later.

Correction:

int &refA = a;

Error 2: Non-const reference bound to an rvalue literal

Incorrect code:

int &refLiteral = 100;

Explanation:

A non-const lvalue reference cannot bind directly to a temporary value or literal.

Correction:

const int &refLiteral = 100;

Error 3: Non-const reference bound to a different data type

Incorrect code:

double pi = 3.14159;
int &refInt = pi;

Explanation:

The variable pi is of type double, whereas refInt is an integer reference. A non-const integer reference cannot bind to a double variable.

Correction:

int refInt = static_cast<int>(pi);

Error 4: Modification through a constant reference

Incorrect code:

const int &constRef = 50;
constRef = constRef + 10;

Explanation:

A constant reference does not allow modification of the value to which it refers.

Correction:

const int &constRef = 50;

The value cannot be changed through constRef.

2. Explain lifetime extension of temporary objects using const references.

When a temporary object is bound to a constant reference, C++ extends the lifetime of that temporary object to match the lifetime of the reference in the applicable local scope.

Example:

const int &ref = 50;

Here:

- The value 50 is a temporary object.
- The constant reference ref binds to it.
- Its lifetime is extended to the lifetime of ref.
- The value cannot be modified through the reference.

3. Corrected C++ Program

```
#include <iostream>
using namespace std;

int main()
{
   int a = 25;

   // Valid reference initialization
   int &refA = a;

   // Constant reference to literal
   const int &refLiteral = 100;

   // Type conversion
   double pi = 3.14159;
   int refInt = static_cast<int>(pi);

   // Constant reference
   const int &constRef = 50;

   cout << "Original Value: " << a << endl;
   cout << "Reference Value: " << refA << endl;
   cout << "Literal Reference: " << refLiteral << endl;
   cout << "Integer Value: " << refInt << endl;
   cout << "Constant Reference: " << constRef << endl;

   return 0;
}
```

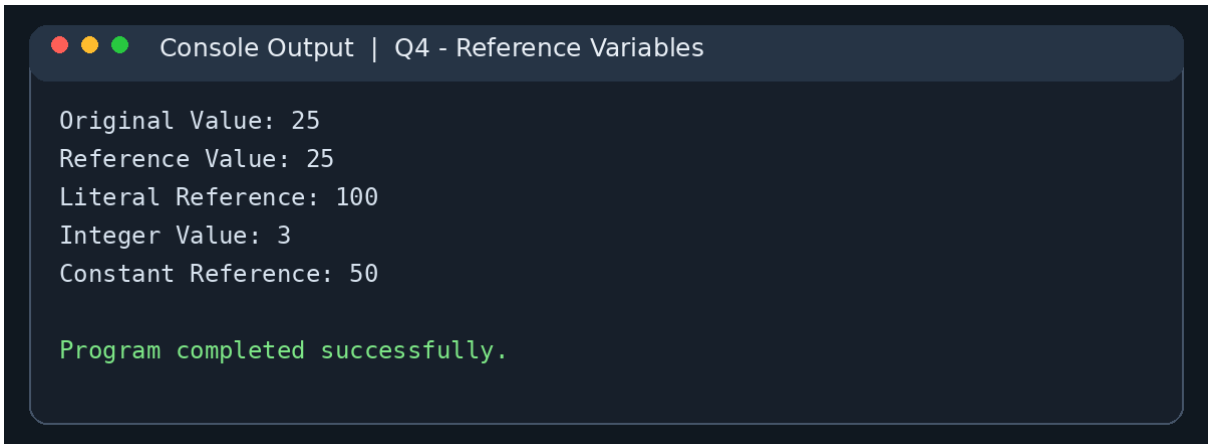
4. Dry Run Table

Step	Variable	Value

1	a	25
2	refA	25
3	refLiteral	100
4	pi	3.14159
5	refInt	3
6	constRef	50

Expected Console Output

Original Value: 25
Reference Value: 25
Literal Reference: 100
Integer Value: 3
Constant Reference: 50



Question 5: Dynamic Array Allocation & Stray Loop Semicolon

Topic: Memory Operators (new, delete), Loops and Off-by-One Boundaries.

1. Explain the consequence of the semicolon at the end of the for statement.

The incorrect statement is:

```
for (int i = 0; i < size; ++i);
```

The semicolon at the end of the for loop creates an empty loop body.

The following block:

```
{
   readings[i] = (i + 1) * 10;
}
```

is treated as a separate block and is not part of the loop.

Therefore, the variable i is not accessible inside that block because it was declared within the for loop.

Correction: Remove the extra semicolon and place the initialization statements inside the loop.

```
for (int i = 0; i < size; ++i)
{
   readings[i] = (i + 1) * 10;
}
```

2. Identify the defect in Line 21. Explain the difference between delete and delete[].

Incorrect statement:

```
delete readings;
```

Explanation:

The array was dynamically allocated using:

```
int *readings = new int[size];
```

When memory is allocated using new[], it must be released using delete[].

- delete is used to release memory allocated for a single object using new.
- delete[] is used to release memory allocated for an array using new[].

Correction:

```
delete[] readings;
```

3. Corrected C++ Program

```
#include <iostream>
using namespace std;

int main()
{
   int size = 5;

   // Dynamic memory allocation
   int *readings = new int[size];

   // Initialize array elements
   for (int i = 0; i < size; ++i)
   {
      readings[i] = (i + 1) * 10;
   }
```

```
    // Calculate total
    int total = 0;

    for (int i = 0; i < size; ++i)
    {
        total += readings[i];
    }

    cout << "Sensor Readings: ";

    for (int i = 0; i < size; ++i)
    {
        cout << readings[i] << " ";
    }

    cout << endl;

    cout << "Total: " << total << endl;

    // Release allocated memory
    delete[] readings;

    return 0;
}
```

4. Dry Run / Trace Table

The array size is 5.

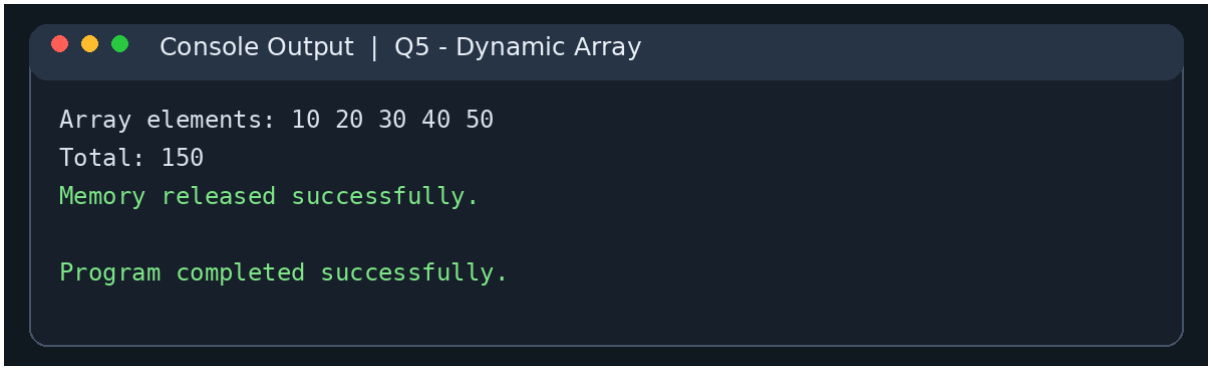
Iteration	i	Calculation	Reading
1	0	(0 + 1) × 10	10
2	1	(1 + 1) × 10	20
3	2	(2 + 1) × 10	30
4	3	(3 + 1) × 10	40
5	4	(4 + 1) × 10	50

Sum Calculation:

$$ Total = 10+20+30+40+50 $$
$$ Total = 150 $$

Expected Console Output

Sensor Readings: 10 20 30 40 50
Total: 150



UNIT 1 – SECTION B: PSEUDO-CODE WRITING QUESTIONS

Question 1: Procedural-to-OOP Paradigm Transformation

Topic: OOP Paradigm, Encapsulation, Data Hiding and Abstraction.

1. Design Overview

The Water Consumer Billing System is designed using Object-Oriented Programming (OOP). It uses a class named WaterConsumer to store consumer details and calculate water bills.

The main concepts used are:

1. Class: WaterConsumer represents a water consumer.
2. Encapsulation: Consumer details and billing information are stored together.
3. Data Hiding: All attributes are declared private.
4. Abstraction: Public methods provide access to billing operations without exposing internal calculations.
5. Validation: Meter readings must be non-negative, and current readings must be greater than or equal to previous readings.

2. Functional Requirements

The system contains the following private attributes:

- consumerID – STRING
- previousReading – REAL
- currentReading – REAL
- unitsConsumed – REAL
- totalBill – REAL

Public methods:

- Initialize()
- CalculateConsumption()
- ComputeBill()
- DisplayBill()

3. Structured Pseudo-Code

```
CLASS WaterConsumer

PRIVATE:
   consumerID : STRING
   previousReading : REAL
   currentReading : REAL
   unitsConsumed : REAL
   totalBill : REAL

PUBLIC:

   METHOD Initialize(id, prev, curr)

      IF prev < 0 OR curr < 0 THEN
         DISPLAY "Invalid Meter Reading"
         RETURN FALSE
      ENDIF

      IF curr < prev THEN
         DISPLAY "Current Reading Cannot Be Less Than Previous Reading"
         RETURN FALSE
      ENDIF

      consumerID = id
      previousReading = prev
      currentReading = curr

      unitsConsumed = 0
      totalBill = 0

      RETURN TRUE

   ENDMETHOD


   METHOD CalculateConsumption()

      unitsConsumed = currentReading - previousReading

   ENDMETHOD


   METHOD ComputeBill()

      REAL bill = 0

      IF unitsConsumed <= 100 THEN
```

```
            bill = unitsConsumed * 1.50

        ELSE IF unitsConsumed <= 300 THEN

            bill = (100 * 1.50) +
                ((unitsConsumed - 100) * 2.50)

        ELSE

            bill = (100 * 1.50) +
                (200 * 2.50) +
                ((unitsConsumed - 300) * 4.00)

            bill = bill + (bill * 0.10)

        ENDIF

        totalBill = bill

    ENDMETHOD


    METHOD DisplayBill()

        DISPLAY "WATER BILL INVOICE"
        DISPLAY "Consumer ID: ", consumerID
        DISPLAY "Previous Reading: ", previousReading
        DISPLAY "Current Reading: ", currentReading
        DISPLAY "Units Consumed: ", unitsConsumed
        DISPLAY "Total Bill: Rs.", totalBill

    ENDMETHOD

ENDCLASS


BEGIN MAIN

    DECLARE consumer : WaterConsumer
    DECLARE id : STRING
    DECLARE prev, curr : REAL
    DECLARE valid : BOOLEAN

    INPUT id
    INPUT prev
    INPUT curr

    valid = consumer.Initialize(id, prev, curr)

    IF valid == TRUE THEN
```

```
        consumer.CalculateConsumption()
        consumer.ComputeBill()
        consumer.DisplayBill()

    ELSE

        DISPLAY "Billing Process Failed"

    ENDIF

END MAIN
```

4. Dry Run / Trace Table

Given Sample Input:

- Previous Reading = 1450.0
- Current Reading = 1820.0
- Consumer ID = C101

Step 1: Calculate Units Consumed

$$ Units = 1820-1450 $$
$$ Units = 370 $$

Step 2: Calculate Tiered Bill

Tariff Slab	Units	Rate	Amount
First 100 units	100	₹1.50	₹150
Next 200 units	200	₹2.50	₹500
Above 300 units	70	₹4.00	₹280
Total	370		₹930

Step 3: Calculate Environmental Surcharge

$$ Surcharge = 930 \times \frac{10}{100} $$
$$ Surcharge = ₹93 $$

Step 4: Calculate Final Bill

$$ TotalBill = 930+93 $$
$$ \boxed{TotalBill = ₹1023} $$

5. Final Output

WATER BILL INVOICE

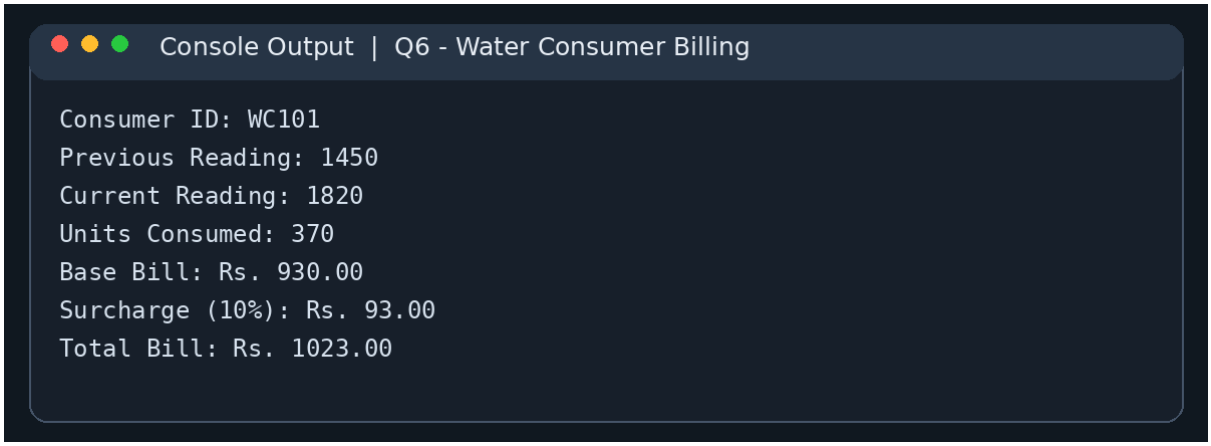
Consumer ID: C101

Previous Reading: 1450
Current Reading: 1820
Units Consumed: 370

First 100 Units: Rs.150
Next 200 Units: Rs.500
Above 300 Units: Rs.280

Basic Bill: Rs.930
Environmental Surcharge (10%): Rs.93

Total Bill: Rs.1023



Question 2: Multi-Tier Tax & Precision Cast Engine

1. Design Overview

The Tax Calculation Engine calculates annual income tax based on gross salary and eligible investment deductions.

- Uses REAL data type for salary and tax calculations.
- Applies deduction limits of ₹1,50,000 and ₹25,000.
- Uses conditional statements to calculate progressive tax.
- Applies 4% Health and Education Cess.
- Converts the final tax into an integer using nearest-rupee rounding.

2. Structured Pseudo-Code

BEGIN

INPUT grossAnnualSalary
INPUT deduction80C
INPUT deductionHealthInsurance

deduction80C = MIN(deduction80C, 150000)
deductionHealthInsurance = MIN(deductionHealthInsurance, 25000)

```
taxableIncome = MAX(0, grossAnnualSalary -
                deduction80C -
                deductionHealthInsurance)

IF taxableIncome <= 250000 THEN
   tax = 0

ELSE IF taxableIncome <= 500000 THEN
   tax = (taxableIncome - 250000) * 0.05

ELSE IF taxableIncome <= 1000000 THEN
   tax = 12500 + (taxableIncome - 500000) * 0.20

ELSE
   tax = 112500 + (taxableIncome - 1000000) * 0.30
ENDIF

finalTax = tax + (tax * 0.04)

roundedTax = INTEGER(finalTax + 0.5)

DISPLAY roundedTax

END
```

3. Dry Run Table

Given Input:

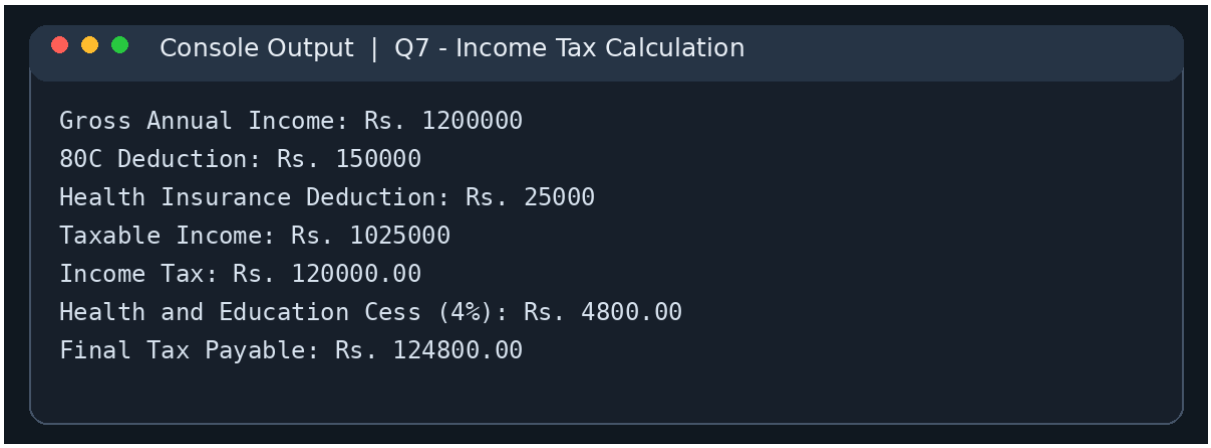
- Gross Salary = ₹12,00,000
- Deduction 80C = ₹1,80,000
- Health Insurance = ₹30,000

Particulars	Amount
Gross Salary	₹12,00,000
80C Deduction (capped)	₹1,50,000
Health Insurance (capped)	₹25,000
Taxable Income	₹10,25,000
Raw Tax	₹1,20,000
4% Cess	₹4,800
Final Tax	₹1,24,800

4. Expected Output

Gross Annual Salary: 1200000
80C Deduction: 150000
Health Insurance Deduction: 25000
Taxable Income: 1025000

Raw Tax: 120000
Health and Education Cess: 4800
Final Tax: 124800



Question 3: Bitwise Mask Validation & Sensor Health Telemetry

1. Design Overview

The UAV Sensor Health System checks the status of different hardware components using bitwise operators.

- Bitwise AND (&) checks sensor status.
- Bitwise OR (|) combines error masks.
- Bitwise XOR (^) identifies missing sensors.
- Bitwise NOT (~) helps identify unset bits.
- Flight is allowed only when all mandatory sensors are active and no critical warnings exist.

2. Bitmask Constants

Sensor	Hexadecimal Mask
Gyroscope	0x01
Accelerometer	0x02
Altimeter	0x04
GPS	0x08
Low Battery	0x10
Over Temperature	0x20
Radio Link	0x40
Hardware Failure	0x80

3. Structured Pseudo-Code

BEGIN CheckFlightReadiness(statusRegister)

MANDATORY = 0x0F

```
INHIBITORS = 0xB0

IF (statusRegister & 0x10) != 0 THEN
   DISPLAY "Low Battery Warning"
ENDIF

IF (statusRegister & 0x20) != 0 THEN
   DISPLAY "Over Temperature Warning"
ENDIF

IF (statusRegister & 0x80) != 0 THEN
   DISPLAY "Fatal Hardware Failure"
ENDIF

missingSensors = MANDATORY XOR
            (statusRegister AND MANDATORY)

IF missingSensors != 0 THEN
   DISPLAY "Missing Sensor Mask: ", missingSensors
ENDIF

IF (statusRegister & MANDATORY) == MANDATORY
  AND (statusRegister & INHIBITORS) == 0 THEN

   DISPLAY "Flight Ready"

ELSE

   DISPLAY "Flight Not Ready"

ENDIF

END
```

4. Dry Run Table

Test 1: Status Register = 0x4F

Particulars	Result
Mandatory sensors	All active
Low battery	No
Over temperature	No
Hardware failure	No
Radio link	Active
Final status	Flight Ready

Output:

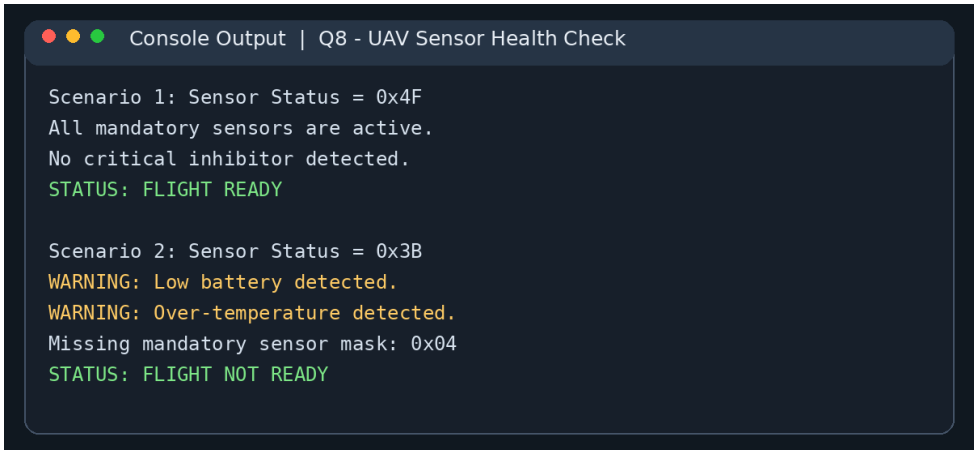
Flight Ready

Test 2: Status Register = 0x3B

Particulars	Result
Gyroscope	Active
Accelerometer	Active
Altimeter	Missing
GPS	Active
Low battery	Warning
Over temperature	Active warning bit
Hardware failure	No
Missing sensor mask	0x04
Final status	Flight Not Ready

Output:

Low Battery Warning
Over Temperature Warning
Missing Sensor Mask: 4
Flight Not Ready



Question 4: Optimized Prime Factorization & Divisor Engine

1. Design Overview

The Divisor Engine finds all proper divisors of a positive integer and classifies it as Perfect, Abundant or Deficient.

- Accepts a positive integer as input.
- Uses loops to identify divisors.
- Uses CONTINUE to skip non-divisors.
- Uses paired divisors to reduce execution time to \(O(\sqrt{N})\).
- Uses conditional statements to classify the number.

2. Structured Pseudo-Code

```
BEGIN

INPUT N

WHILE N <= 0 DO
   DISPLAY "Invalid Input"
   INPUT N
ENDWHILE

IF N == 1 THEN
   DISPLAY "Deficient Number"
   END
ENDIF

sumOfDivisors = 1

FOR d = 2 TO SQRT(N) DO

   IF N MOD d != 0 THEN
      CONTINUE
   ENDIF

   sumOfDivisors = sumOfDivisors + d

   pairedDivisor = N / d

   IF pairedDivisor != d THEN
      sumOfDivisors = sumOfDivisors + pairedDivisor
   ENDIF

ENDFOR

IF sumOfDivisors == N THEN
   DISPLAY "Perfect Number"

ELSE IF sumOfDivisors > N THEN
   DISPLAY "Abundant Number"

ELSE
   DISPLAY "Deficient Number"

ENDIF

END
```

3. Dry Run Table

Test Case 1: N = 28

Divisor (d)	Paired Divisor	Sum
Initial	1	1
2	14	17
3	Not divisible	17
4	7	28

Final sum = 28

Output:

28 is a Perfect Number

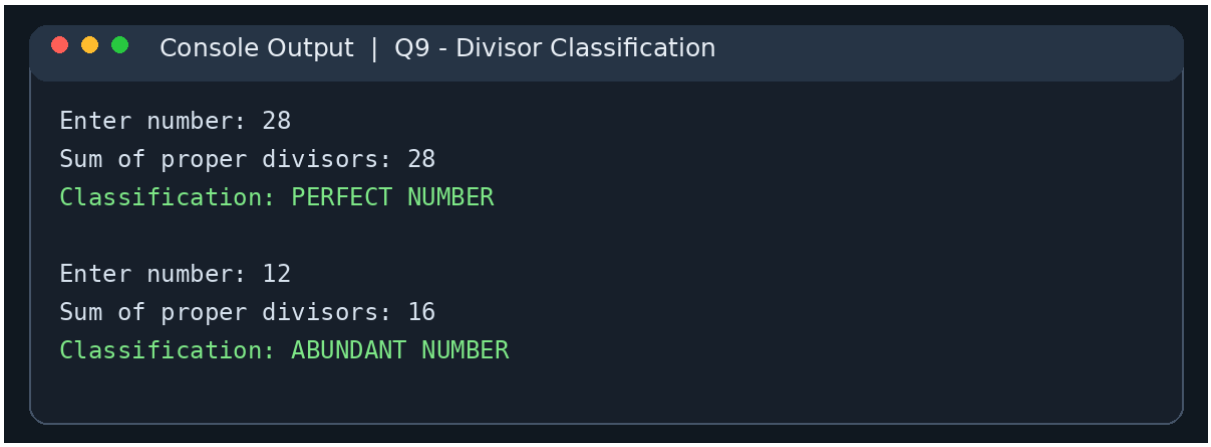
Test Case 2: N = 12

Divisor (d)	Paired Divisor	Sum
Initial	1	1
2	6	9
3	4	16

Final sum = 16

Output:

12 is an Abundant Number



Question 5: State-Machine ATM Controller with Sentinel Loops

1. Design Overview

The ATM Transaction Management System performs banking operations securely using a PIN authentication system and a menu-driven interface.

- Uses a 4-digit PIN for authentication.
- Allows a maximum of 3 login attempts.
- Provides balance inquiry, deposit and withdrawal options.
- Uses switch-case to select transactions.

- Uses while loop to continue transactions until the user exits.
- Validates withdrawal amount and available balance.

2. Structured Pseudo-Code

```
BEGIN ATM_SessionController

SET correctPIN = 4821
SET balance = 5000
SET attempts = 0
SET authenticated = FALSE

WHILE attempts < 3 DO

   INPUT PIN

   IF PIN == correctPIN THEN
      authenticated = TRUE
      BREAK
   ELSE
      attempts = attempts + 1
      DISPLAY "Incorrect PIN"
   ENDIF

ENDWHILE

IF authenticated == FALSE THEN
   DISPLAY "Card Locked"
   END
ENDIF

WHILE TRUE DO

   DISPLAY "1. Balance Inquiry"
   DISPLAY "2. Cash Deposit"
   DISPLAY "3. Cash Withdrawal"
   DISPLAY "4. Exit"

   INPUT choice

   SWITCH choice

      CASE 1:
         DISPLAY balance
         BREAK

      CASE 2:
         INPUT amount

         IF amount > 0 THEN
```

```
            balance = balance + amount
            DISPLAY "Deposit Successful"
         ELSE
            DISPLAY "Invalid Amount"
         ENDIF
         BREAK

      CASE 3:
         INPUT amount

         IF amount > 0 AND amount <= balance THEN

            IF amount MOD 100 == 0 THEN
               balance = balance - amount
               DISPLAY "Withdrawal Successful"
            ELSE
               DISPLAY "Amount Must Be Multiple of 100"
            ENDIF

         ELSE
            DISPLAY "Invalid Amount or Insufficient Balance"
         ENDIF
         BREAK

      CASE 4:
         DISPLAY "Thank You"
         EXIT LOOP

      DEFAULT:
         DISPLAY "Invalid Option"

   ENDSWITCH

ENDWHILE

END
```

3. Dry Run / Test Trace

Initial Balance = ₹5,000

Step	Operation	Input	Result	Balance
1	Enter PIN	1234	Incorrect PIN	₹5,000
2	Enter PIN	4821	Login Successful	₹5,000
3	Select Withdrawal	3	Withdrawal selected	₹5,000
4	Enter Amount	₹250	Rejected (not multiple of 100)	₹5,000
5	Select Withdrawal	3	Withdrawal selected	₹5,000
6	Enter Amount	₹300	Successful	₹4,700

7	Select Exit	4	Session terminated	₹4,700

4. Expected Output

Enter PIN: 1234
Incorrect PIN

Enter PIN: 4821
Login Successful

1. Balance Inquiry
2. Cash Deposit
3. Cash Withdrawal
4. Exit

Enter Choice: 3
Enter Withdrawal Amount: 250

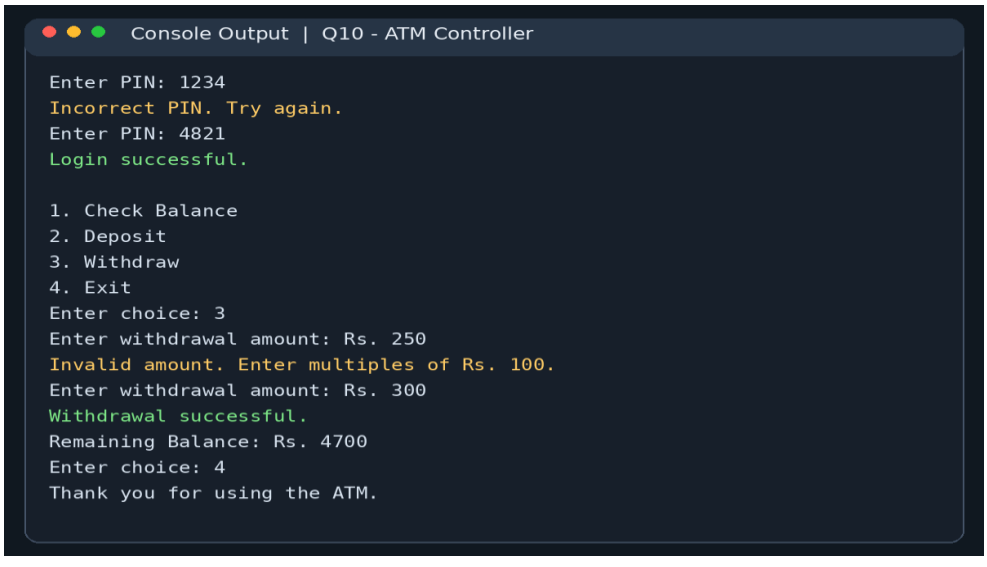
Amount Must Be Multiple of 100
Transaction Declined

Enter Choice: 3
Enter Withdrawal Amount: 300

Withdrawal Successful

Remaining Balance: 4700

Enter Choice: 4
Thank You
Session Terminated



5. Conclusion

The ATM Controller uses authentication, conditional statements, switch-case and sentinel-controlled loops to perform banking transactions. It ensures secure access, validates deposits and withdrawals, and prevents invalid transactions.